Unit-4

Asp.net Core

- ASP.NET is a popular web-development framework for building web apps on the .NET platform.
- ASP.NET Core is the open-source version of ASP.NET, that runs on macOS, Linux, and Windows. ASP.NET Core was first released in 2016 and is a re-design of earlier Windows-only versions of ASP.NET.

Web forms

- ASP.NET Web Forms is a part of the ASP.NET web application framework and is included with Visual Studio.
- It is one of the four programming models you can use to create ASP.NET web applications, the others are ASP.NET MVC, ASP.NET Web Pages, and ASP.NET Single Page Applications.
- Web Forms are pages that your users request using their browser. These pages can be written using a combination of HTML, client-script, server controls, and server code.
- When users request a page, it is compiled and executed on the server by the framework, and then the framework generates the HTML markup that the browser can render. An ASP.NET Web Forms page presents information to the user in any browser or client device.

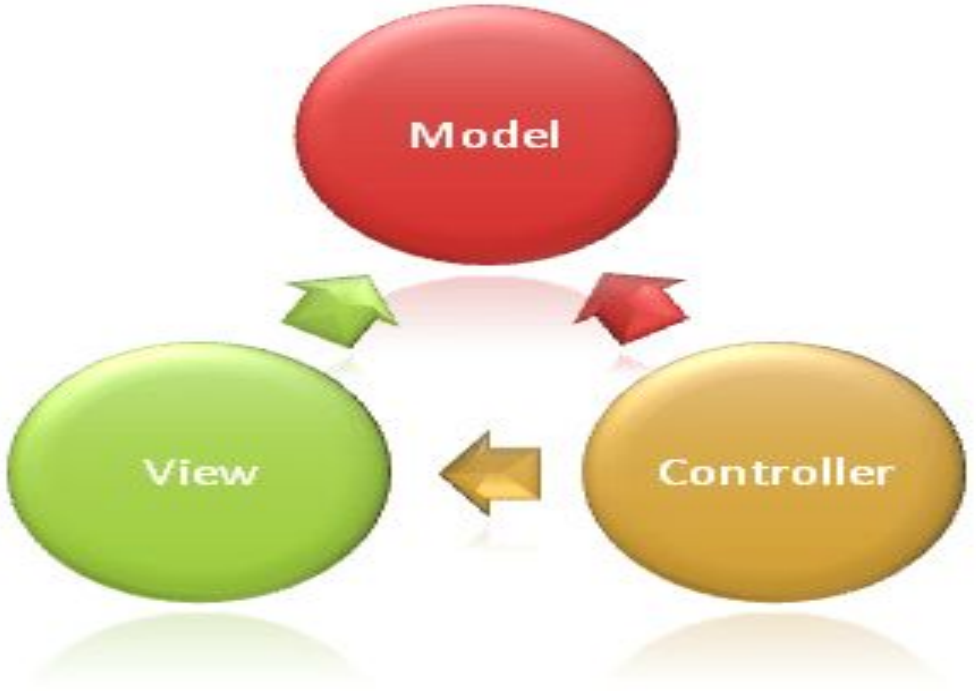
Web pages

- Web Pages is one of many programming models for creating ASP.NET web sites and web applications.
- Web Pages provides an easy way to combine HTML, CSS, and server code:
- Easy to learn, understand, and use
- Uses an SPA application model (Single Page Application)
- Similar to PHP and Classic ASP
- VB (Visual Basic) or C# (C sharp) scripting languages

ASP.NET Core MVC

- The Model-View-Controller (MVC) architectural pattern separates an application into three main groups of components: Models, Views, and Controllers.
- This pattern helps to achieve separation of concerns. Using this pattern, user requests are routed to a Controller which is responsible for working with the Model to perform user actions and/or retrieve results of queries.
- The Controller chooses the View to display to the user, and provides it with any Model data it requires.
- The following diagram shows the three main components and which ones reference the others:

ASP.NET Core MVC



Model

- The Model in an MVC application represents the state of the application and any business logic or operations that should be performed by it.
- Business logic should be encapsulated in the model, along with any implementation logic for persisting the state of the application.
- Strongly-typed views typically use ViewModel types designed to contain the data to display on that view. The controller creates and populates these ViewModel instances from the model.

View

- Views are responsible for presenting content through the user interface.
- They use the Razor view engine to embed .NET code in HTML markup. There should be minimal logic within views, and any logic in them should relate to presenting content. If you find the need to perform a great deal of logic in view files in order to display data from a complex model, consider using a View Component, ViewModel, or view template to simplify the view.

Controller

- Controllers are the components that handle user interaction, work with the model, and ultimately select a view to render. In an MVC application, the view only displays information; the controller handles and responds to user input and interaction.
- In the MVC pattern, the controller is the initial entry point, and is responsible for selecting which model types to work with and which view to render (hence its name - it controls how the app responds to a given request).

2-tier, 3-tier and n-tier architecture in asp.net core

- In software development, including ASP.NET Core, the term "tier" refers to the logical separation of components in a system.
- These tiers are categorized based on their functionality and the way they interact with each other. The most common architectures are 2-tier, 3-tier, and n-tier architectures.

2-Tier Architecture

- Also known as the client-server architecture.
- It consists of two main components: the client (presentation layer) and the server (data layer).
- In an ASP.NET Core application, the client could be a web browser or a desktop application, and the server is typically a database server.
- This architecture is simple but can become difficult to maintain as the application grows, as there is no clear separation between the presentation and business logic.

2-Tier Architecture

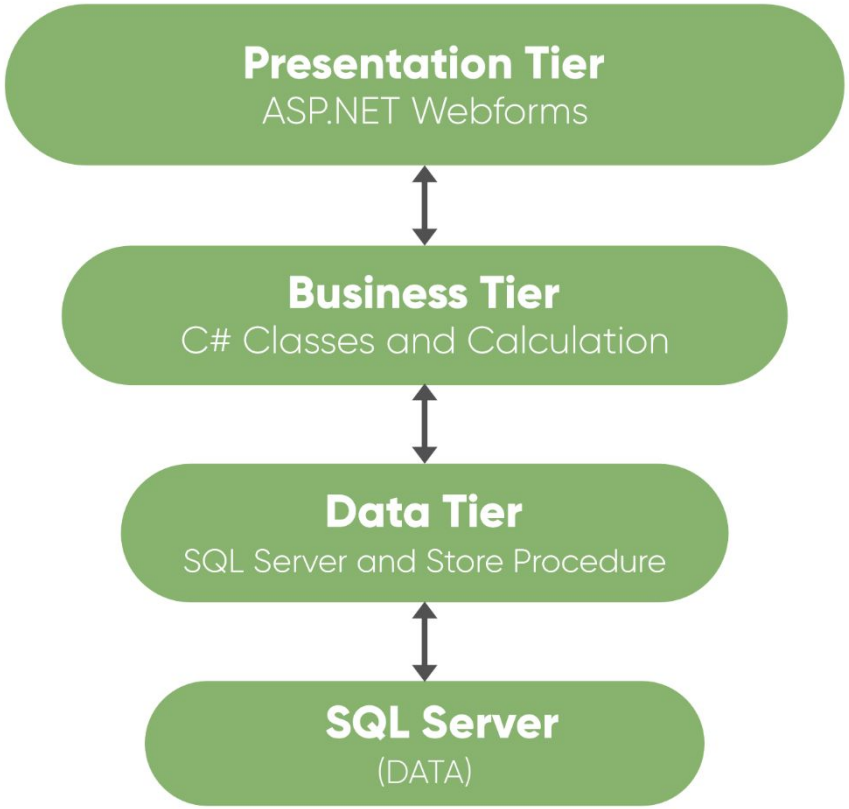
2-Tier Architecture

- In two-tier, the application logic is either buried inside the User Interface on the client or within the database on the server (or both). With two-tier client/server architectures, the user system interface is usually located in the user's desktop environment and the database management services are usually in a server that is a more powerful machine that services many clients.

3-Tier Architecture

- In three-tier, the application logic or process lives in the middle-tier, it is separated from the data and the user interface.
- Three-tier systems are more scalable, robust and flexible.
- In addition, they can integrate data from multiple sources. In the three-tier architecture, a middle tier was added between the user system interface client environment and the database management server environment.
- There are a variety of ways of implementing this middle tier, such as transaction processing monitors, message servers, or application servers.

3-Tier Architecture



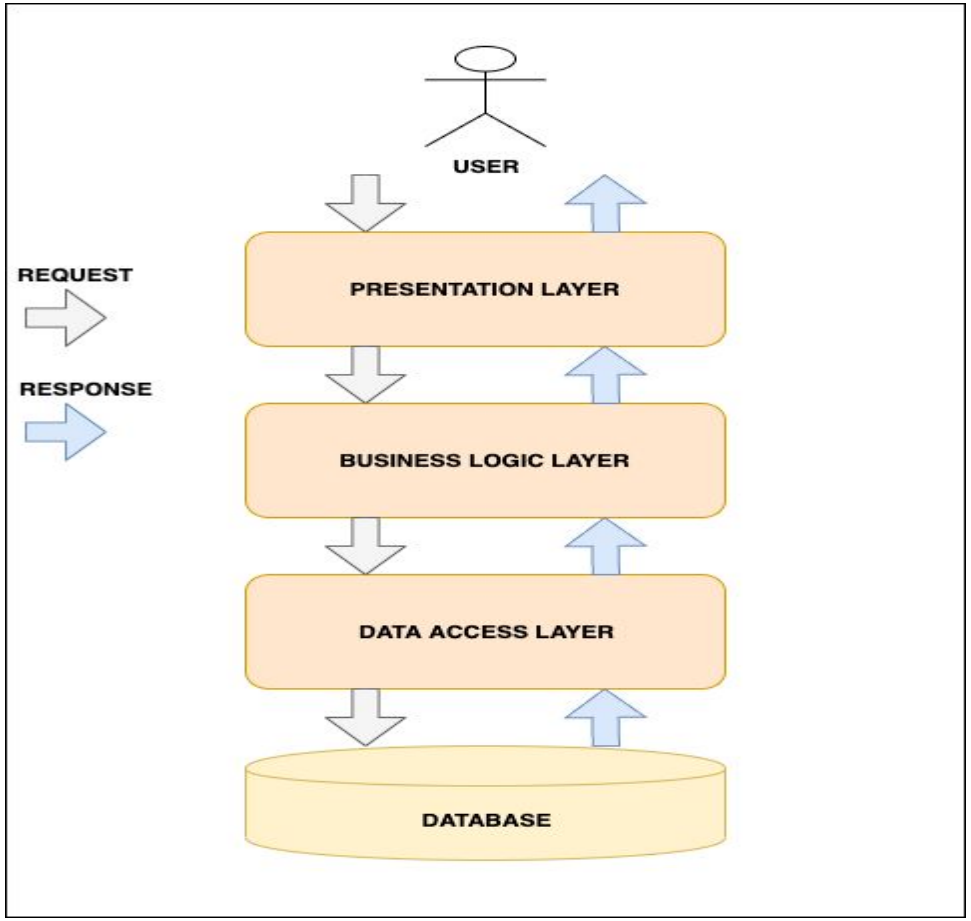
3-Tier Architecture Advantages

- Updation to new graphical environments is easier and faster.
- Easy to maintain large and complex projects.
- It provides logical separation between presentation layer, business layer and data layer.
- By introducing application layer between data and presentation layer, it provides more security to database layer hence data will be more secure.
- You can hide unnecessary methods from the business layer in the presentation layer.
- Data provider queries can be easily updated and OOP’s concept can be easily applied on project.

N-Tier Architecture

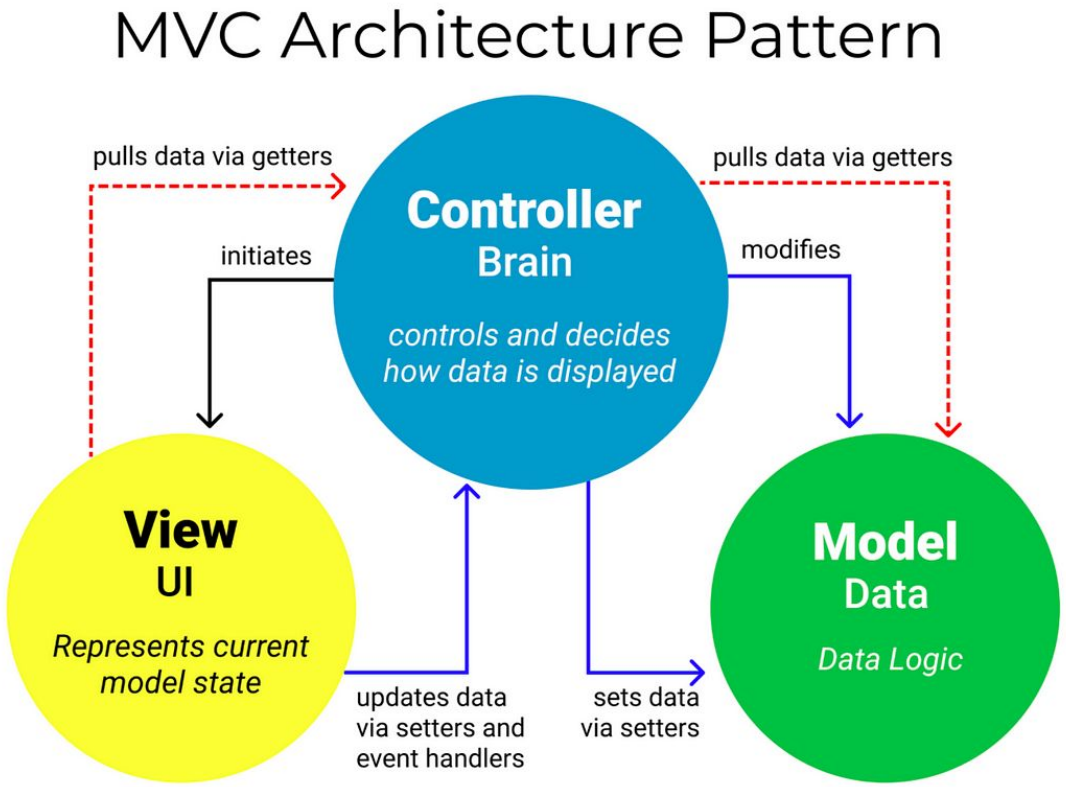
- An extension of the 3-tier architecture, where the application is divided into multiple layers or tiers to handle specific functionalities.
- Common tiers include Presentation, Application, Business, Data, Services, and more depending on the complexity of the application.
- In ASP.NET Core, n-tier architectures are designed to achieve greater modularity and flexibility. For example, you might have separate layers for authentication, caching, logging, etc.
- Each tier focuses on specific concerns, making it easier to manage and scale the application.

N-Tier Architecture



MVC architectural pattern

- The Model-View-Controller (MVC) architectural pattern separates an application into three main groups of components: Models, Views, and Controllers.



MVC architectural pattern

- **Model:**
- The Model represents the data and business logic of the application.
- It is responsible for managing and processing data, as well as defining the rules and behaviors of the application.
- In the context of ASP.NET Core, the Model often corresponds to classes that represent data entities, as well as business logic components like services.

MVC architectural pattern

- **View:**
- The View is responsible for presenting the user interface and displaying data to the user.
- It receives user input and sends it to the Controller for further processing.
- In ASP.NET Core, the View is typically implemented using Razor syntax in .cshtml files, defining the structure and layout of the user interface.

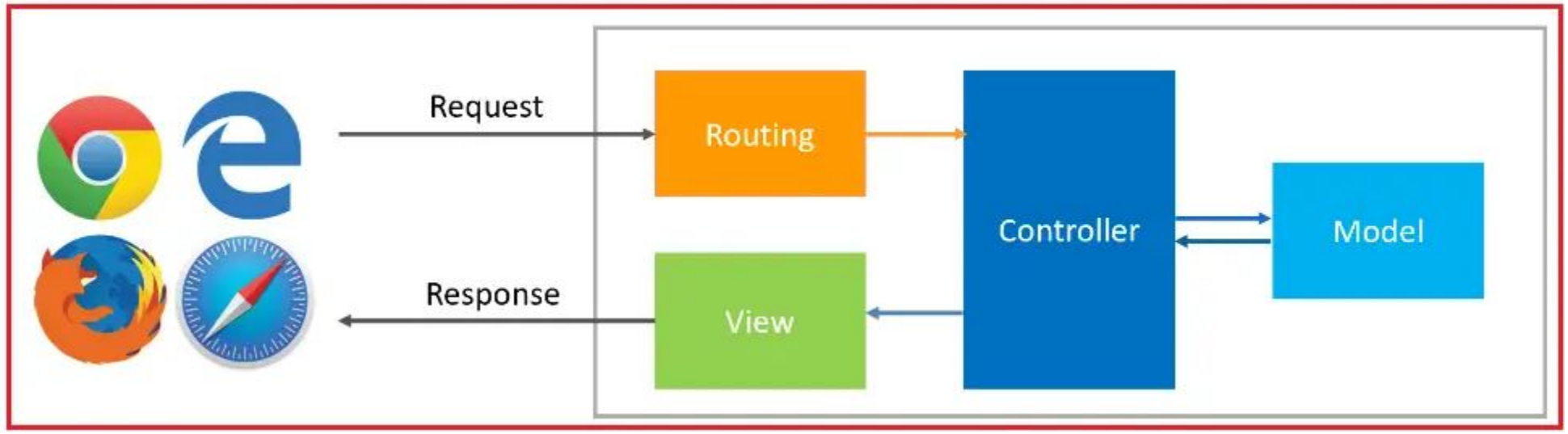
MVC architectural pattern

- **Controller:**
- The Controller acts as an intermediary between the Model and the View.
- It handles user input, processes requests from the View, and updates the Model accordingly.
- In ASP.NET Core, controllers are classes that contain action methods responsible for responding to user requests, interacting with the Model, and determining which View to render.

URL Routing in ASP.NET Core MVC

- Routing in the ASP.NET Core MVC Web Application is a mechanism that will inspect the incoming HTTP requests (i.e., URLs) and then map that HTTP request to the controller's action method.
- This mapping is done by the routing rules which are defined for the application.
- We can do this by adding the Routing Middleware to the Request Processing Pipeline.
- You can configure multiple routes for your application, and for each route, you can also set some specific configurations such as default values, constraints, etc.

URL Routing in ASP.NET Core MVC



URL Routing in ASP.NET Core MVC

- When the client makes a request, i.e., an HTTP Request, then that request is first received by the routing engine.
- Once the Routing engine receives an HTTP Request, the Routing system tries to find out the matching route pattern of the requested URL with already registered routes.
- Routes contain information about the Controller name, action method name, method type (Get, Post, Put, Patch, Delete), method parameters, route data, etc.

URL Routing in ASP.NET Core MVC

- If it finds a matching URL pattern for the incoming request, then it forwards the request to the appropriate controller and action method. If no match for the incoming HTTP request URL Pattern is found, it returns a 404 HTTP status code to the client.

URL Routing in ASP.NET Core MVC

- **Different Types of Routing**

1. **Convention-Based Routing**
2. **Attribute-Based Routing.**

Convention-Based Routing

- Conventional routing was the original routing approach used in ASP.NET Core MVC. It uses a predefined URL pattern to determine which controller and action method to execute.
- This pattern is defined in the Program.cs file and typically specifies placeholders for the controller and action names.
- **Configuration:** It is defined globally in the application's startup configuration, meaning that the routing rules are applied universally.
- **Usage:** This approach is useful for applications with a clear and simple URL structure, where the URLs follow a consistent pattern.
- **Example:** Defining a route pattern like {controller=Home}/{action=Index}/{id?} would match a URL like /Products/Details/5 and map it to the Details action on the Products controller with 5 as an action parameter.

Attribute-Based Routing

- Attribute Routing in ASP.NET Core MVC is a powerful approach to defining URL routes by using Route Attributes directly on controller actions or controllers themselves.
- With Attribute-Based Routing, you can specify the routing configuration at a very granular level, making routing logic more explicit and closely tied to action methods. This approach provides a way to explicitly specify the route for each controller and action.
- **Configuration:** Routes are defined using attributes on controllers and action methods. This means each controller or action can have its own routing defined, making it easier to understand and maintain the routing rules.
- **Usage:** This is particularly useful for complex URL structures and APIs or when you need routes that don't easily fit into a conventional pattern.
- **Example:** Using **[Route("products/details/{id?}")]** on an action method would map the specific URL **/products/details/5** to that action.

URL Routing in ASP.NET Core MVC

- In ASP.NET Core MVC, URL rewriting can be achieved using the Rewrite middleware. The middleware allows you to rewrite or redirect incoming URLs based on predefined rules.
- Var newurl = new rewriteoptions().addrewrite(“newpagename”,”oldpagename”,false)
- In the AddRedirect method, specify the old-url and new-url parameters to redirect requests from the old URL to the new URL. You can also use the AddRewrite method to rewrite URLs based on a pattern and provide a replacement URL.

- You can add multiple AddRedirect and AddRewrite rules to handle different URL rewriting scenarios.

Models in ASP.NET Core

- A model is a class with .cs (for C#) extension with properties and methods. Models are used to set or get the data. If your application does not have data, then there is no need for a model. If your application has data, then you need a model.
- The Models in ASP.NET Core MVC Application contain a set of classes representing the domain data (you can also say the business data) and logic to manage the domain/business data.
- So, in simple words, we can say that the Model is the component in the MVC Design pattern that is used to manage the data, i.e., the state of the application in memory. The Model represents a set of classes used to describe the application's validation, business, and data access logic.

Models in ASP.NET Core

- **Add a data model class**
- Right-click the Models folder > Add > Class. Name the file Movie.cs.
- public class Movie
- {
- public int Id { get; set; }
- public string? Title { get; set; }
- [DataType(DataType.Date)]
- public DateTime ReleaseDate { get; set; }
- public string? Genre { get; set; }
- public decimal Price { get; set; }
- }

Controller in ASP.NET Core

- A Controller is a special class in ASP.NET Core Application with .cs (for C# language) extension.
- By default, you can see the Controllers reside in the Controllers folder when you create a new ASP.NET Core Application using the Model View Controller (MVC) Project Template.
- In the ASP.NET Core MVC Application, the controller class should and must be inherited from the **Controller** base class.

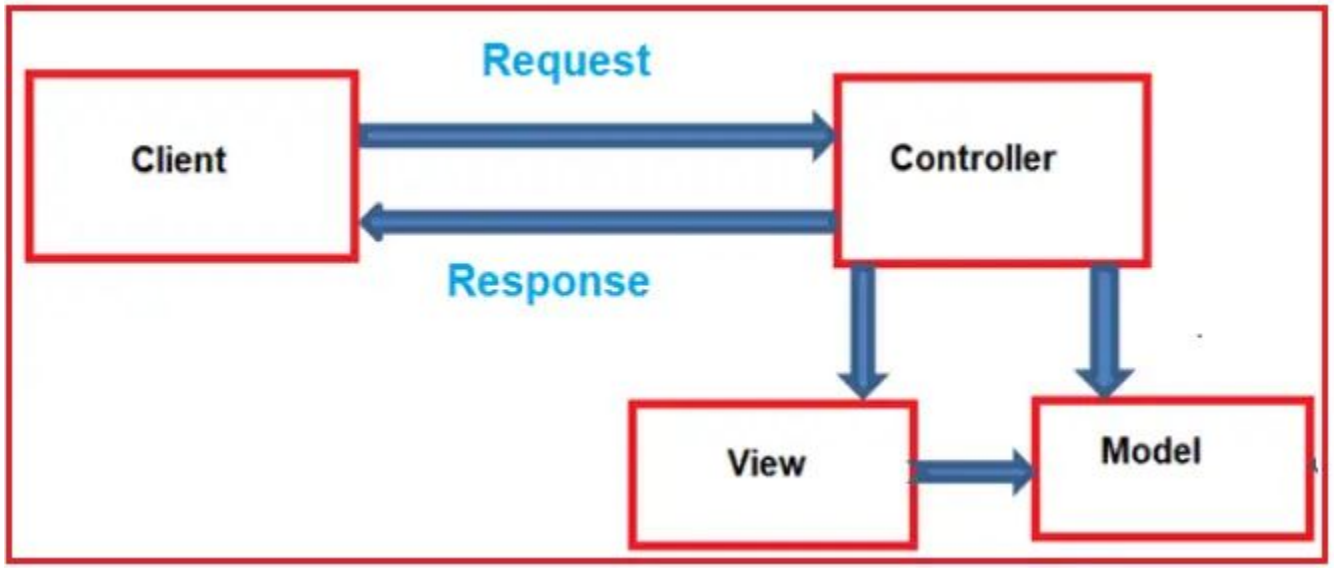
Controller in ASP.NET Core

- Controllers in the MVC Design Pattern are the components that handle the user interaction, work with the model, and ultimately select a view to render.
- In the MVC Design Pattern, the controller is the initial entry point and is responsible for selecting which model types to work with and which view to render.
- The Controllers in the ASP.NET Core MVC Application logically group similar types of actions together. This grouping of similar types of action together allows us to define sets of rules, such as caching, routing, and authorization, which will be applied collectively.

Role of Controller in ASP.NET Core

- When the client (browser) sends a request to the server, then that request first goes through the request processing pipeline.
- Once the request passes the request processing pipeline, it will hit the controller. Inside the controller, there are lots of methods (called action methods) that actually handle the incoming HTTP Requests.
- The action method inside the controller executes the business logic and prepares the response, which is then sent back to the client who initially made the request.

Role of Controller in ASP.NET Core



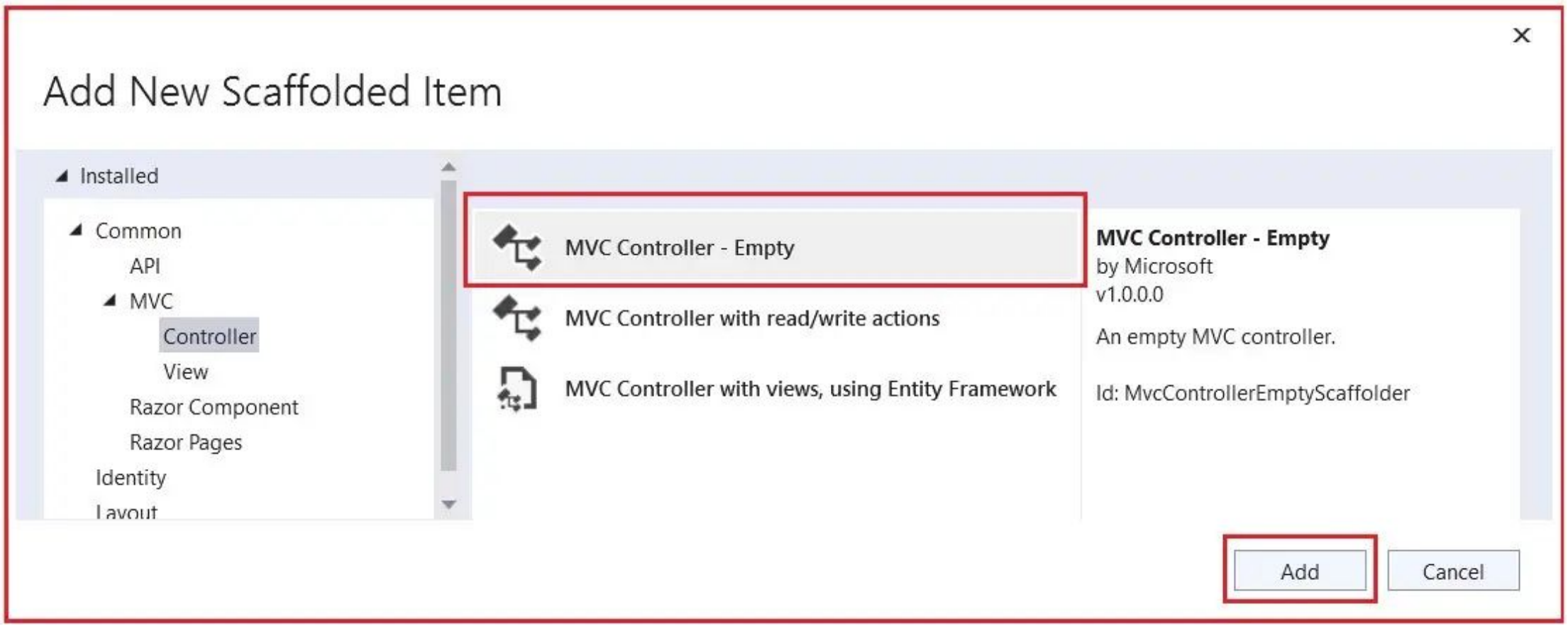
Role of Controller in ASP.NET Core

- **Group Related Actions:** A Controller groups related actions, i.e., Action Methods.
- **Request Handling:** Controllers receive HTTP requests and decide how to process them.
- **Features Applied:** Many important features, such as Caching, Routing, Security, etc., can be applied to the controller.
- **Model Interaction:** They often interact with models to retrieve data or save changes.
- **View Selection:** Controllers select a view and provide it with any necessary data for rendering.

Creating Controller

- If you create the ASP.NET Core Application using the MVC Project Template, then it will create HomeController within the Controllers folder by default.
- we need to add a controller (StudentController) inside this Controllers folder. To do so, right-click on the Controller folder and select the **Add => Controller** option from the context menu, which will open the Add Controller window,

Creating Controller



Creating Controller

- Once you click on the **Add** button, it will open the below window where you need to select the **Controller Class – Empty** option and give a meaningful name to your controller. Here, I give the name as **StudentController** and click on the **Add** button. The Controller name should be suffixed with the word Controller.

Working with Views

- In the Model-View-Controller (MVC) pattern, the View is the component that contains logic to represent the model data (the model data provided to it by a controller) as a user interface with which the end-user can interact.
- The Views in ASP.NET Core MVC are HTML templates with embedded Razor markup, which generate content that is sent to the client.
- A view in ASP.NET Core MVC Application is a file with a ".cshtml" (for C# language) extension. The meaning of **cshtml = CS (C Sharp) + HTML**.
- That means the view is a combination of programming language (C#) and HTML (Hypertext Mark-up Language). The Views in the ASP.NET Core MVC Application are generally returned from the Controller Action Method.

What is the Role of View in the ASP.NET Core MVC Application?

- A view in ASP.NET Core MVC Application is responsible for UI, i.e., application data presentation.
- That means we display information about the website on the browser using the views only.
- A user generally performs all the actions on a view, such as a button click, form, list, and other UI elements.
- Views in the MVC Application are responsible for presenting content through the user interface. In the ASP.NET Core MVC Application, we use the Razor View Engine to embed .NET code in HTML markup.
- There should be minimal logic within views, and any logic in them should only be related to presenting the content.

Create a New View

- In the Views folder, create a folder named Home (matching the controller name). Inside the Home folder, add a new view file named Index.cshtml.
- The content of Index.cshtml can be a simple HTML markup:
- <!DOCTYPE html>
- <html>
- <head>
- <title>My ASP.NET Core MVC View</title>
- </head>
- <body>
- <h1>Hello, ASP.NET Core MVC!</h1>
- </body>
- </html>

Rendering a Partial View

- **Render the View from the Action Method:**
- public class HomeController : Controller
- {
- public IActionResult Index()
- {
- return View();
- }
- }

customview with and without model

- In ASP.NET Core MVC, you can create custom views both with and without a model.
- **Custom View Without Model:**
- Step 1: Create a new View:
- Inside the Views folder, create a folder named Shared (for shared views) or any other folder of your choice. Inside that folder, add a new view file. For example, CustomViewWithoutModel.cshtml.

customview with and without model

- <!-- CustomViewWithoutModel.cshtml -->
- <!DOCTYPE html>
- <html>
- <head>
- <title>Custom View Without Model</title>
- </head>
- <body>
- <h1>This is a Custom View Without Model</h1>
- No model data is required for this view.
- </body>
- </html>

customview with and without model

- **Step 2: Use the View:** You can use this view in a controller action by returning it directly:
- public class HomeController : Controller
- {
- public IActionResult Index()
- {
- return View("Shared/CustomViewWithoutModel");
- }
- }

customview with and without model

- **Custom View With Model:**
- Step 1: Create a Model:
- Create a model class. For example, CustomViewModel.cs:
- // CustomViewModel.cs
- public class CustomViewModel
- {
- public string Message { get; set; }
- }

customview with and without model

- Step 2: Create a new View:
- Inside the Views folder, create a folder for the controller (e.g., Home). Inside that folder, add a new view file. For example, CustomViewWithModel.cshtml.
- <!-- CustomViewWithModel.cshtml -->
- @model CustomViewModel

- <!DOCTYPE html>
- <html>
- <head>
- <title>Custom View With Model</title>
- </head>
- <body>
- <h1>@Model.Message</h1>
- This is a Custom View With Model.
- </body>
- </html>

customview with and without model

- **Step 3:Use the View with Model:**
- In the controller, create an instance of the model and pass it to the view:
- public class HomeController : Controller
- {
- public IActionResult Index()
- {
- var model = new CustomViewModel
- {
- Message = "Hello from the model!"
- };
- return View("Home/CustomViewWithModel", model);
- }
- }

layout views and partial views

- In ASP.NET Core MVC, layout views and partial views are two distinct concepts used to structure the user interface of your web application. Let's go over each one:
- **Layout View:**
- A layout view provides the overall structure and common elements for multiple pages in your application. It usually contains the HTML structure, headers, footers, navigation menus, and other shared elements. In ASP.NET Core MVC, the layout view is typically stored in the Views/Shared folder and is often named _Layout.cshtml.
- **Partial View:**
- A partial view is a reusable component that can be included in multiple views or layout views. It's useful for breaking down complex UIs into smaller, manageable components. Partial views are often stored in the Views/Shared folder and are named with the prefix _ (underscore) to indicate that they are partial views.
- layout views provide the overall structure for your application, and partial views allow you to modularize and reuse specific components within your views. These concepts help maintain a clean and organized code structure in your ASP.NET Core MVC application.

layout view example

```
<!DOCTYPE html>
<html>
<head>
    <title>@ViewData["Title"] - My ASP.NET Core App</title>
</head>
<body>
    <header>
        <h1>My ASP.NET Core App</h1>
        <nav> <ul> <li><a asp-controller="Home" asp-action="Index">Home</a></li></ul> </nav>
    </header>
    <div>
        @RenderBody()
    </div>
</body>
</html>
```

partial view example

- <!-- _MyPartialView.cshtml -->

-
- <h2>Partial View Content</h2>
- <!-- Add partial view content here -->
-
- To use a partial view in another view or layout view, you can use the Partial helper:

partial view example

- <!-- SomeView.cshtml -->
- @{
- ViewData["Title"] = "Some Page";
- }
- <h1>Some Page</h1>

- @await Html.PartialAsync("_MyPartialView")

HTMLhelper class, extension and strongly type methods

- In ASP.NET Core MVC, the HtmlHelper class is a fundamental component that assists in generating HTML markup in views. It provides methods for rendering HTML controls, URLs, and other elements in a type-safe manner.
- HtmlHelper Class:
- The HtmlHelper class is part of the Microsoft.AspNetCore.Mvc.Rendering namespace. It is accessible in views and helps generate HTML elements.

HTMLhelper class, extension and strongly type methods

- @{
- // Example of using HtmlHelper to generate a hyperlink
- var link = Html.ActionLink("Home", "Index", "Home");
- } @link
- In the above example, Html.ActionLink is a method of HtmlHelper that generates an HTML anchor (<a>) element based on the specified controller, action, and parameters.

HTMLhelper class, extension and strongly type methods

- **Extension Methods:**
- You can create your own extension methods for the HtmlHelper class to encapsulate custom functionality. These extension methods should be placed in a static class, typically in the Helpers folder.
- // Example of an extension method
- public static class CustomHtmlHelperExtensions
- {
- public static IHtmlContent CustomButton(this IHtmlHelper helper, string buttonText)
- {
- var button = new TagBuilder("button");
- button.InnerHtml.Append(buttonText);
- return button;
- }
- }

HTMLhelper class, extension and strongly type methods

- Now, you can use this extension method in your views:
- @{
- // Example of using a custom extension method
- var customButton = Html.CustomButton("Click me");
- }

- @customButton

HTMLhelper class, extension and strongly type methods

- **Strongly Typed Methods**

- Strongly typed methods in ASP.NET Core MVC allow you to work with models directly in your views. By passing a model to a view, you can use strongly typed methods to generate HTML controls based on the properties of the model.
- @model MyViewModel
- <!-- Example of using strongly typed methods -->
- <form asp-action="SubmitForm" asp-controller="Home" method="post">
- <label asp-for="Name"></label>
- <input asp-for="Name" />
- <label asp-for="Age"></label>
- <input asp-for="Age" />
- <button type="submit">Submit</button>
- </form>
- asp-for is a strongly typed method that generates HTML attributes based on the properties of the MyViewModel model.

Design of view using CSS

- Designing views in an ASP.NET Core MVC application involves using HTML and CSS to create a visually appealing and responsive user interface. Here are several ways you can enhance the design of views using CSS in ASP.NET Core MVC:

1. Separate CSS Files:

- Keep your CSS rules in separate files for better organization.
- Place these files in the wwwroot/css directory, and link to them in your views.
- Example in your Razor view file (e.g., Views/Home/Index.cshtml):
- <link rel="stylesheet" href="~/css/site.css" />

Design of view using CSS

2. Responsive Design:

- Use media queries to create responsive designs that adapt to different screen sizes.
- @media screen and (max-width: 600px) {
- /* Styles for small screens */
- }

3. Bootstrap or Other CSS Frameworks:

- Utilize CSS frameworks like Bootstrap for pre-built components and a responsive grid system.
- Include Bootstrap in your project and reference it in your HTML.
- <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />

Design of view using CSS

4. Flexbox and Grid Layouts:

- Leverage Flexbox and CSS Grid for creating flexible and responsive layouts.
- .container {
- display: flex;
- justify-content: space-between;
- }

5. SASS or LESS:

- Use SASS or LESS for more advanced CSS features, such as variables and mixins.
- Compile these into regular CSS files.

Design of view using CSS

6. Custom Styles for ASP.NET Core Tag Helpers:

- Apply custom styles to ASP.NET Core Tag Helpers to enhance their appearance.
- Example for a form input:
- input[type="text"] {
- width: 100%;
- padding: 0.5em;
- }

Design of view using CSS

7. CSS Transitions and Animations:

- Add transitions and animations to create smooth effects.
- .fade-in {
- opacity: 0;
- transition: opacity 1s ease-in-out;
- }

- .fade-in.show {
- opacity: 1;
- }

use of java script and bootstrap in asp.net core mvc

- JavaScript and Bootstrap are commonly used in ASP.NET Core MVC applications to enhance the user interface and provide a more dynamic and responsive user experience.
- **JavaScript:**
- **Client-Side Validation:** JavaScript can be used for client-side form validation to provide immediate feedback to users before submitting data to the server. This can improve the user experience by reducing the number of round trips to the server.
- **AJAX (Asynchronous JavaScript and XML):** JavaScript can be employed to make asynchronous requests to the server, enabling partial page updates without requiring a full page reload. This is often used for updating portions of a page dynamically.

use of java script and bootstrap in asp.net core mvc

- **Bootstrap:**
- **Responsive Design:** Bootstrap is a popular front-end framework that provides a responsive grid system and components. It helps create web applications that look good and function well on various devices and screen sizes.
- **UI Components:** Bootstrap offers a collection of pre-designed UI components like navigation bars, buttons, forms, modals, and more. These components can be easily integrated into ASP.NET Core MVC views to speed up the development process.
- **Grid System:** Bootstrap's grid system allows you to create responsive layouts, making it easier to design pages that adapt to different screen sizes, from desktops to mobile devices.

use of java script and bootstrap in asp.net core mvc

```
<!DOCTYPE html>
<html>
<head>
    <title>My ASP.NET Core MVC App</title>
    <!-- Bootstrap CSS -->
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" integrity="...">
</head>
<body>
    <div class="container">
        @RenderBody()
    </div>
    <!-- Bootstrap JS and Popper.js (required for some Bootstrap components) -->
    <script src="https://code.jquery.com/jquery-3.3.1.slim.min.js" integrity="..."></script>
    <script src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.9.1/dist/umd/popper.min.js" integrity="..."></script>
    <script src="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/js/bootstrap.min.js" integrity="..."></script>
</body>
</html>
```

Working with Razor pages

- In ASP.NET Core MVC, "Razor" refers to a markup syntax for combining server-side C# code with HTML to create dynamic web pages.
- Razor syntax provides a concise and expressive way to generate HTML content dynamically based on data from your application.
- Razor pages in ASP.NET Core MVC are similar to traditional ASP.NET Web Forms or MVC views but provide a more page-focused approach to building web applications.
- Razor pages allow you to define a page model (similar to a controller in MVC) and a Razor view in a single file.
- This makes it easier to manage the logic and presentation of a particular page within a single file.

Working with Razor pages

- @page
- @model HelloWorldModel
- <!DOCTYPE html>
- <html>
- <head>
- <title>Hello World</title>
- </head>
- <body>
- <h1>Hello, @Model.Name!</h1>
- </body>
- </html>

Data Validation

- Data validation is a crucial aspect of building secure and reliable ASP.NET Core MVC applications. It helps ensure that the data entered by users is accurate, consistent, and meets the expected criteria. Here are some common approaches to perform data validation in ASP.NET Core MVC:
- 1. Model Validation:
- ASP.NET Core MVC supports model validation by using data annotations on your model properties. You can apply attributes like [Required], [StringLength], [Range], etc., to specify validation rules. For example:

Data Validation

- public class Product
- {
- [Required(ErrorMessage=“*”]
- public string Name { get; set; }

- [Range(1, 100,ErrorMessage=“range must be in between 1 to 100”)]
- public int Price { get; set; }
- }
- **In controller**
- if (ModelState.IsValid)
- { // Valid data, process it // ... }
- else {
- // Invalid data, return the view with validation errors
- return View(product);
- }

Data Validation

2. Custom Validation Attributes:

- You can create custom validation attributes by implementing the ValidationAttribute class. This allows you to define custom validation rules. For example:
- public class CustomDateValidationAttribute : ValidationAttribute
- {
- protected override ValidationResult IsValid(object value, ValidationContext validationContext)
- {
- DateTime date = (DateTime)value;
- if (date < DateTime.Now)
- {
- return ValidationResult.Success;
- }
- else
- {
- return new ValidationResult("Date must be in the past.");
- }
- }
- }

Data Validation

In Model

```
public class MyModel
{
    [CustomDateValidation]
    public DateTime SomeDate { get; set; }
}
```

3. Client-Side Validation:

Consider using client-side validation in combination with server-side validation for a better user experience. You can enable client-side validation by including the necessary JavaScript libraries and using the appropriate HTML helpers in your views.

Managing temporary data using models asp.net core

- TempData is used to transfer data from view to controller, controller to view, or from one action method to another action method of the same or a different controller.
- TempData stores the data temporarily and automatically removes it after retrieving a value.
- TempData is a property in the ControllerBase class. So, it is available in any controller or view in the ASP.NET MVC application.

Managing temporary data using models asp.net core

- The TempData in ASP.NET MVC is a mechanism for transmitting a small amount of temporary data from one controller to one view and one controller method of action to another method of action either inside the same controller or with another controller.
- The value of TempData will be null when the next query is completed by default. But if you want, you can change that default behavior too.

Managing temporary data using models asp.net core

- **public class** StudentController : Controller
- {
- **public** ActionResult Method1()
- {
- TempData["Name"] = "Ifour";
- TempData["Age"] = 21;
- **return** View();
- }
- **public** ActionResult Method2()

Managing temporary data using models asp.net core

- **public** ActionResult Method2()
- {
- **string** Name;
- int Age;
- **if** (TempData.ContainsKey("Name"))
- Name = TempData["Name"].ToString();
- **if** (TempData.ContainsKey("Age"))
- Age = int.Parse(TempData["Age"].ToString())
- **return** View();
- }

Manging data using ado.net

- The **ASP.NET CORE** framework defines a number of namespaces to interact with a **Relational Database System** like Microsoft SQL Server, Oracle, MySQL, etc. Collectively, these namespaces are known as **ADO.NET**.
- ADO.NET Components
- What are the ADO.NET components? ADO.NET components perform specific database related tasks. These are 6 main components which are described below:
- Data Providers – used for communication with the data source.
- Connection – maintains the location information of the data source.
- Command – executes a given command type.
- DataReader – reading data in connected environment.
- DataAdapter – reading data in disconnected environment.

Manging data using ado.net

- The 4 most common Data Providers in ADO.NET are:
- **1. Data Provider for SQL Server** : Provides data access for Microsoft SQL Server Database. It uses System.Data.SqlClient.
- **2. Data Provider for OLE DB** : Provides data access for OLE DB data sources. These are used to access various COM objects and DBMS that does not have a specific .NET data provider. It uses the System.Data.OleDb namespace.
- **3. Data Provider for ODBC** : The ODBC Data Providers are defined within the System.Data.Odbc namespace are typically useful only if we need to communicate with a given DBMS for which there is no custom .NET data provider.
- **4. Data Provider for Oracle** : Provides data access for Oracle databases. Uses the System.Data.OracleClient namespace.

Manging data using ado.net

Object	Description
Connection	Used for connecting and disconnecting from the database. The base class of all Connection objects is DbConnection.
Command	Executes an SQL Command against a database by using connection and transaction objects. The base class of all Command objects is DbCommand.
DataReader	Reads a forward-only, read-only stream of data from a database. The base class for all DataReader objects is the DbDataReader class.
DataAdapter	Used to populate a dataset or datatable with the data from the database. The base class is DbDataAdapter.

Stored Procedures in ASP.NET Core

- Stored procedures are pieces of reusable code that you can save in a database data dictionary.
- It help you extract, edit, and remove data from a database without writing the code to do so again and again. They save you time, reduce workloads, and increase productivity.

Stored Procedures in ASP.NET Core

- CREATE PROCEDURE [dbo].[insertemp]
- @id int = 0,
- @name varchar(20),
- @contact varchar(20),
- @email varchar(20)

- AS
- insert into tblemp(id,name,contact,email)
- values(@id,@name,@contact,@email);
- RETURN 0

Stored Procedures in ASP.NET Core

- To use stored procedures with ADO.NET in a .NET MVC application, you primarily work within the data access layer (DAL) of your application, using classes like SqlConnection and SqlCommand to interact with the database.
- The MVC framework (Controllers, Views, Models) handles the presentation and business logic, while the DAL handles the data interaction.

- **Advantages**
- Improved performance due to precompiled execution
- Enhanced security through parameterized queries
- Centralized database logic
- Easier maintenance and debugging
- Reduced SQL injection risks

Stored Procedures in ASP.NET Core

- **Insert**
- con.Open();
- cmd.CommandType = CommandType.StoredProcedure;
- cmd.CommandText = "insertemp";
- cmd.Connection = con;
- cmd.Parameters.AddWithValue("@id", e.id);
- cmd.Parameters.AddWithValue ("@name", e.name);
- cmd.Parameters.AddWithValue("@contact", e.contact);
- cmd.Parameters.AddWithValue("@email", e.email);
- cmd.ExecuteNonQuery();

Stored Procedures in ASP.NET Core

- **View**
- con.Open();
- cmd.CommandType= CommandType.StoredProcedure;
- cmd.CommandText = "viewemp";
- cmd.Connection = con;
- adp = new SqlDataAdapter(cmd);
- adp.Fill(dt);